

Reviewer Pack — Free Cumulant Gap in Read-Twice BP for IP_2

Entry 6e9e752ae070 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 03:38:01 UTC

Free Cumulant Gap in Read-Twice BP for IP_2

Entry ID: 6e9e752ae070 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 03:38:01 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For a read-twice BP P computing IP_2 , the fourth free cumulant $\kappa_4(\mu_P)$ of its transition matrix eigenvalue distribution satisfies $\kappa_4(\mu_P) \geq \Omega(n)$, while for general read-twice BPs computing constant-depth circuits, $\kappa_4(\mu_P) \leq O(\log \text{size}(P))$

Rationale (proposer’s reasoning):

Free cumulants capture non-commutative dependencies in BP transition matrices. IP_2 ’s exponential lower bound stems from its inherent correlation structure, which should manifest as divergent free cumulants compared to structured circuits with polynomial size

Taxonomy category: BP_READTWICE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 8e6720b4863dadb5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from typing import List, Dict

def run_trial(seed: int) -> Dict[str, any]:
    random.seed(seed)

    def power_method(matrix: List[List[float]], max_iter=1000, tol=1e-6) -> List[float]:
        n = len(matrix)
        x = [random.random() for _ in range(n)]
        x /= sum(x)

        for _ in range(max_iter):
            y = matrix_multiply(matrix, x)
            y_norm = sum(y[i] ** 2 for i in range(n)) ** 0.5
            if abs(y_norm - 1) < tol:
                return y / y_norm
            x = y

        raise ValueError("Power method did not converge")

    def matrix_multiply(A: List[List[float]], B: List[List[float]]) -> List[List[float]]:
        n, m, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(n)]

        for i in range(n):
            for j in range(p):
                for k in range(m):
                    C[i][j] += A[i][k] * B[k][j]

        return C

    def free_cumulant(eigenvalues: List[float], n: int) -> float:
        if len(eigenvalues) < 4:
            raise ValueError("Not enough eigenvalues to compute fourth cumulant")

        mu1 = sum(eigenvalues) / n
        mu2 = sum((e - mu1) ** 2 for e in eigenvalues) / n
        mu3 = sum((e - mu1) ** 3 for e in eigenvalues) / n
        mu4 = sum((e - mu1) ** 4 for e in eigenvalues) / n

        return (mu4 - 3 * mu2 ** 2) / mu2

    def generate_read_twice_BP(n: int, seed: int) -> List[List[float]]:
        random.seed(seed)
```

```

BP = [[0] * n for _ in range(n)]

for i in range(n):
    for j in range(n):
        if i == j:
            BP[i][j] = 1
        else:
            BP[i][j] = random.random()

return BP

def IP2_BP(n: int) -> List[List[float]]:
    BP = [[0] * n for _ in range(n)]

    for i in range(n):
        for j in range(n):
            if i == j:
                BP[i][j] = 1
            else:
                BP[i][j] = 0

    return BP

def eigenvalues(matrix: List[List[float]]) -> List[float]:
    n = len(matrix)
    identity = [[int(i == j) for j in range(n)] for i in range(n)]

    # Compute matrix - λI
    lambda_range = (-10, 10)
    while True:
        lambda_ = random.uniform(*lambda_range)
        A = [[matrix[i][j] - lambda_ * identity[i][j] for j in range(n)] for i in range(n)]

        # Compute determinant using Gaussian elimination
        det = 1.0
        for i in range(n):
            pivot = None
            for j in range(i, n):
                if A[j][i] != 0:
                    pivot = j
                    break

            if pivot is None:
                raise ValueError("Matrix is singular")

            if pivot != i:
                A[i], A[pivot] = A[pivot], A[i]
                det *= -1

            det *= A[i][i]
            for j in range(i + 1, n):
                A[j][i] /= A[i][i]

            for j in range(i + 1, n):
                for k in range(i + 1, n):
                    A[j][k] -= A[j][i] * A[i][k]

```

```

        if abs(det) > 1e-6:
            break

    # Find eigenvalues using bisection method
    def f(lambda_):
        return det - math.exp(-lambda_)

    lambda_min, lambda_max = 0, 20
    while lambda_max - lambda_min > 1e-6:
        lambda_mid = (lambda_min + lambda_max) / 2
        if f(lambda_mid) * f(lambda_min) < 0:
            lambda_max = lambda_mid
        else:
            lambda_min = lambda_mid

    return [lambda_min, lambda_max]

n = 40
IP2_BP_matrix = IP2_BP(n)
read_twice_BPs = [generate_read_twice_BP(n, i) for i in range(30)]

IP2_eigenvalues = eigenvalues(IP2_BP_matrix)
IP2_cumulant = free_cumulant(IP2_eigenvalues, n)

other_BPs = [power_method(read_twice_BP) for read_twice_BP in read_twice_BPs]
other_cumulants = [free_cumulant(eigenvalues(bp), n) for bp in other_BPs]

return {
    "metric_name": "Free Cumulant",
    "metric_value": IP2_cumulant,
    "instances_tested": 31, # Including the trivial BP
    "conjecture_holds": IP2_cumulant >= n and all(cum <= math.log(n) for cum in other_cumulants),
    "counterexample": "" if IP2_cumulant >= n else "read-twice BP with lower cumulant"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)) ** 0.5
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)

```

```

counterexample = next((r["counterexample"] for r in results if not r["conjecture_holds"])),
print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4b37d8d.py", line 157, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4b37d8d.py", line 138, in run_trial
    IP2_cumulant = free_cumulant(IP2_eigenvalues, n)
                    ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b4b37d8d.py", line 48, in free_cumulant
    raise ValueError("Not enough eigenvalues to compute fourth cumulant")
ValueError: Not enough eigenvalues to compute fourth cumulant

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to insufficient eigenvalues for cumulant computation, preventing result validation | next: Verify eigenvalue sampling adequacy in test setup

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	107829
2	propose	ollama_remote	qwen3:8b	0	0	26947
3	preregistration	ollama_remote	qwen3:8b	0	0	9017
4	novelty	ollama_remote	qwen3:8b	0	0	5852
5	novelty	ollama_remote	qwen3:8b	0	0	36741
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19083
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17579
8	judge	ollama_remote	qwen3:8b	0	0	11210

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 234259 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/6e9e752ae070.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6e9e752ae070.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6e9e752ae070.tar.gz` (if generated)